

OmegaOS Orchestration V3

Lois, règles, skills, Oracles, vérification, rmux et défaut Codex

Synopsis

Audit et refonte de l'orchestration OmegaOS, fondés sur des preuves runtime et sur les recommandations primaires OpenAI, Anthropic et MCP. Le baseline était compris entre 31 et 35 sur 100 selon trois lectures indépendantes. La release candidate atteint 93 sur 100 sur le périmètre vérifié. Elle ne revendique pas 100 sur 100 tant que les parcours live Claude et Gemini, le push final et la session distante Vibe ne sont pas tous prouvés. Cette session utilise GPT-5.6 Sol, pas un modèle Opus Sol.

Résultat exécutif

Le système passe d'une doctrine riche mais fail-open à un noyau compact, typé et vérifiable.

Score release candidate: 93/100. Le score mesure le périmètre réellement exécuté, pas une promesse.

- **Baseline règles et hooks:** 34/100.
- **Baseline lifecycle des skills:** 31/100.
- **Baseline runtime Oracle et worker:** 35/100.
- **Automatisation vérifiée:** 651 tests Rust, 6 tests Python, runner d'audit shell, build workspace, smokes Codex, rmux et sync.
- **Contexte actif:** 10 784 octets après sync, sous le plafond OmegaOS de 24 Ko.
- **Catalogue:** 231 skills natifs, 232 installés avec Agent Reach, 232 activés pour Codex.
- **Audits:** 23 audits issus d'un registre TOML unique.

Le progrès majeur est le changement de frontière de confiance: un worker propose un résultat, puis une vérification indépendante décide de l'accepter. Un message convaincant ou un fichier `done` ne suffit plus.

Les sept lois

Aucune loi n'est inutile dans son intention. Une formulation a été corrigée pour supprimer une contradiction budgétaire.

- **L0, Ship the truth:** un changement doit être reproductible dans la cible et appuyé par des preuves fraîches. Statut: conserver.
- **L1, Runtime is the only truth:** la source et les commentaires décrivent une intention, le runtime tranche. Statut: conserver.
- **L2, Researcher, not sycophant:** contester une prémisse fausse et documenter les hypothèses. Statut: conserver.
- **L3, Decide and proceed:** un agent dispatché choisit la meilleure voie sûre au lieu d'attendre dans une session sans opérateur. Statut: conserver avec les gates d'autorité.
- **L4, Done means 100 percent verified:** tous les livrables demandés doivent être suivis et vérifiés. Statut: conserver.
- **L5, Quality floor within budget:** la qualité est un plancher, mais le temps, le coût, le risque et les tokens sont des budgets explicites. Statut: formulation révisée. L'ancien texte disait que les ressources étaient illimitées et contredisait R-BUDGET.
- **L6, Finish the mission:** énumérer, planifier, exécuter, vérifier et rapporter, sans abandon silencieux. Statut: conserver.

La hiérarchie est désormais explicite: les lois restent universelles, mais elles opèrent sous l'autorité du host, du système et de l'utilisateur.

Règles universelles et qualité

Neuf règles, conservées avec une meilleure frontière entre doctrine et enforcement.

- **R-STACK, Stack defaults:** conserver comme politique de projet, charger selon le domaine.
- **R-KARPATY, quatre principes de code:** conserver, utile comme heuristique transversale.
- **R-CLI, préférer la CLI au MCP:** conserver comme préférence, pas comme interdiction d'une intégration nécessaire.
- **R-RUBRIC, rubric before execution:** conserver et lier aux critères du plan typé.
- **R-VERIFY, adversarial verification:** conserver et renforcer par la décision indépendante d'acceptation.
- **R-BUDGET, mission budget:** conserver comme borne de coût, tokens et retries.
- **R-PROD, prod verification:** conserver pour tout changement déployé.
- **R-TEST, layered testing:** règle révisée. Les tests unitaires et d'intégration précèdent les previews et le golden path de production. La production n'est plus le premier laboratoire pour une mutation risquée.
- **R-LOOP, bounded retries:** conserver, avec plafonds et escalade humaine.

Verdict: aucune suppression. Les règles R-VERIFY, R-BUDGET et R-LOOP doivent être des contraintes runtime, pas seulement du texte.

Règles d'orchestration, première partie

Dix règles couvrent le plan, la topologie, les audits et le développement produit.

- **R-PLAN:** conserver, projeter le DAG typé vers le task tool de chaque provider.
- **R-ORCH:** conserver, mais choisir la topologie la plus simple qui satisfait la mission. Le multi-agent n'est pas une fin.
- **R-GOAL:** conserver uniquement pour Claude, car `/goal` est un mécanisme provider-specific.
- **R-MASTER:** conserver la séparation discussion, classification et dispatch.
- **R-AUDIT:** conserver. Un audit nommé exécute son vrai protocole.
- **R-SKILL-ATLAS:** conserver, avec `SkillCatalogV1` comme source unique pour Atlas, RAG et activation provider.
- **R-PRODUCT:** conserver sur les décisions de produit, charger à la demande.
- **R-BLUEPRINT-STACK:** conserver pour les nouveaux AgentikOS, indexer le long runbook au lieu de l'injecter partout.
- **R-STREAM:** conserver comme runbook de transport rmux, chargé uniquement pour une mission stream.
- **R-MONITOR:** conserver comme runbook de surveillance, chargé uniquement pour une mission monitor.

Le changement clé est le scoping. Une règle pertinente reste forte sans consommer le contexte de toutes les missions.

Règles d'orchestration, seconde partie

Neuf règles spécialisées restent disponibles sans polluer le noyau global.

- **R-COUNCIL:** conserver pour les décisions irréversibles, contestées ou architecture-wide.
- **R-MARKETING:** conserver comme routeur spécialisé à la demande.
- **R-VISUAL-ID:** conserver pour la paire Higgsfield Soul et génération.
- **R-ZERNIO:** conserver pour les publications et ads des comptes OmegaOS.
- **R-ZERNFLOW:** conserver pour les flows inbound et DM.
- **R-BROWSER:** conserver la séparation Playwright déterministe et browser-use agentique.
- **R-TRINITY:** conserver pour la sécurité white-hat, sous les limites du host et du périmètre autorisé.
- **R-MODEL:** conserver uniquement pour Claude dans sa forme actuelle. Le moteur provider-neutral sélectionne d'abord les capacités requises.
- **R-DESIGN:** conserver comme routeur de skills design, chargé par domaine.

Verdict: ces règles ne sont pas inutiles. Ce qui était inutile était leur présence intégrale dans le contexte de missions sans rapport.

Règles de reporting et sécurité

Dix-neuf règles protègent la preuve, les livrables, les secrets et les opérations à risque.

Reporting

- **R-CITE:** conserver, toute conclusion d'audit porte une preuve.
- **R-STYLE:** conserver, langue utilisateur et code en anglais.
- **R-NODASH:** conserver pour le contenu visible et marketing.
- **R-PDF:** conserver, le moteur `omega pdf` reste la source unique.
- **R-SKILLPUB:** conserver pour les nouveaux skills publiables.
- **R-ARTIFACT:** conserver le routeur à trois surfaces.
- **R-HTML:** conserver pour le fichier autonome offline.
- **R-TGDELIVER:** conserver pour envoyer chaque lien ou fichier à l'opérateur.

Sécurité

- **R-SCOPE, R-SYNC, R-ENV, R-TGSEC:** conserver sans changement de principe.
- **R-VERCEL:** conserver pour le déploiement headless, charger sur missions Vercel.
- **R-SEC:** conserver comme autorisation interne, toujours subordonnée aux limites du host et au scope réel.
- **R-FICHE, R-PROJ:** conserver pour empêcher toute confusion de coordonnées ou de secrets.
- **R-REPO-INSTALL:** conserver la revue supply-chain minimale avant exécution.
- **R-SECRETS-VAULT:** conserver, charger pour les workflows de credentials et recovery.
- **R-DESTRUCT:** conserver la gate explicite avant toute opération irréversible.

Ce qui était devenu obsolète

*La mise à jour moderne des agents ne supprime pas le besoin de règles.
Elle change où elles doivent vivre et comment elles sont appliquées.*

Quatre éléments étaient obsolètes ou contre-productifs:

- **Le texte L5 sur les ressources illimitées.** Il a été remplacé par un plancher de qualité sous budget explicite.
- **Le test production-only.** R-TEST est maintenant une stratégie en couches, avec production comme preuve finale d'un changement déployé.
- **Les mécanismes Claude injectés dans Codex et Gemini.** R-GOAL et R-MODEL sont provider-scoped. Le noyau actif reste provider-neutral.
- **Les longs runbooks globaux.** Stream, monitor, marketing, design, Vercel et autres domaines restent disponibles, mais leur corps complet est chargé uniquement lorsqu'une mission le justifie.

Mon avis: supprimer les règles serait une erreur. Les modèles récents réduisent certains échecs de raisonnement, pas les erreurs de coordination, d'autorité, de reprise, de preuve ou de side effects. La bonne évolution est de compiler les règles pertinentes, de mesurer le contexte et de transformer les invariants critiques en code.

Mission ledger et frontière de confiance

Le ledger devient l'autorité d'écriture. Les fichiers historiques deviennent des projections.

La release candidate ajoute un ledger SQLite WAL pour les événements de mission, les versions de plan, les attempts, les leases clôturés par fencing token et l'outbox transactionnelle (`crates/omega-core/src/mission_ledger.rs:285`).

Chaque mutation porte `expected_version` et `idempotency_key` . Une écriture obsolète échoue au lieu d'écraser l'état (`mission_ledger.rs:509-563`). Les side effects sont enregistrés dans la même transaction que l'événement (`mission_ledger.rs:647-648`).

La complétion suit désormais trois objets distincts:

- `EvidenceClaim` , ce que le worker affirme.
- `EvidenceObservation` , ce que le verifier observe.
- `AcceptanceDecision` , ce que le moteur accepte.

Le modèle est compatible avec les recommandations modernes: un manager peut utiliser des agents comme tools ou effectuer un handoff, les tours sont bornés, les approvals risqués sont sérialisables, et les traces sont des événements de premier rang.

Un catalogue, un registre, des états explicites

La découverte, le RAG, l'installation et l'activation provider dérivent du même catalogue.

`SkillCatalogV1` compile récursivement les racines OmegaOS, normalise les chemins, rejette les collisions et produit un digest déterministe (`crates/omega-core/src/skill_registry.rs:294`). Les métadonnées de compatibilité absentes conservent temporairement les providers existants avec warning, afin d'éviter une migration qui désactive silencieusement 230 skills.

Le packaging `pdfgen` a été réparé dans cette mission: son `SKILL.md` est validé, installé et activé pour Codex. Après sync, le runtime expose 231 skills natifs et 232 skills installés avec Agent Reach.

Les 23 audits sont lus depuis `skills/audits/registry.toml` par Rust et par le runner shell (`crates/omega-core/src/audit.rs:115`). Le runner refuse une collecte incomplète, une evidence invalide, un score sous le seuil ou un verdict final mal formé. Un résultat d'audit incomplet ne peut plus être présenté comme PASS.

Vue rmux pour Codex et OpenAI

La vue conserve les couleurs, recompose les lignes longues et garde le curseur visible.

La nouvelle couche `preview.rs` recompose les lignes par grapheme plutôt que par octet, ce qui protège les accents, les emoji et les caractères larges (`crates/omega-tui/src/preview.rs:29`). Elle remappe le curseur après wrap (`preview.rs:77`), conserve les styles ANSI et corrige les foregrounds illisibles sur fond explicite.

Le provider est détecté à partir de l'identité de session et des signaux de footer (`preview.rs:160`). La vue affiche un accent Codex stable, même après resize.

Preuve runtime: une vraie session Codex a reçu une phrase française plus large que le panneau sans soumission. Le texte est resté visible, a été réparti sur trois lignes, le curseur est resté affiché et le label `CODEx` a été rendu. Les 83 tests TUI passent, dont les cas couleurs, wrap, graphemes, curseur et resize.

Codex devient le défaut sans écraser les choix

Les nouvelles sessions OmegaOS et Oracle choisissent Codex, tandis qu'un provider explicitement configuré est conservé.

Les configurations fraîches et les champs absents utilisent `codex` (`crates/omega-core/src/config.rs:104-109`). Un choix explicite Claude, Gemini ou Codex n'est jamais migré silencieusement (`config.rs:569-572`). La configuration active contient `agent_command = "codex"` et `aisb_agent = "codex"`.

Le smoke réel a lancé Codex 0.146.0 avec GPT-5.6 Sol et effort xhigh. Cette session elle-même est GPT-5.6 Sol. L'expression « Opus Sol » mélange deux familles de modèles et n'est pas l'identité du runtime actuel.

Le contexte agent synchronisé est provider-neutral et mesure 10 784 octets. Les 54 entrées constitutionnelles, soit 7 lois et 47 règles, restent disponibles, tandis que seules les mécaniques pertinentes sont injectées en détail.

Matrice de validation runtime

Chaque résultat important a une commande ou un artefact reproductible.

- **Rust workspace:** 9 tests CLI, 1 test d'intégration Codex, 558 tests core, 83 tests TUI. Total 651, zéro échec.
- **Hooks:** 12 fixtures Claude, Codex et Gemini exercées par 2 tests Python.
- **Catalogue:** 4 tests Python, test de fermeture Rust, digest stable.
- **Audit runner:** protocole shell PASS avec finalisation fail-closed.
- **Build:** `cargo build --workspace` PASS.
- **Diff:** `git diff --check` PASS.
- **Install parity:** toutes les gates fonctionnelles passent. Avant le commit final, l'unique échec attendu est le worktree non commité.
- **Sync isolé:** 231 skills natifs, ajout externe préservé, 232 liens provider dans l'installation active.
- **Règles:** contexte déterministe sous 24 Ko et lois jamais supprimées par le scoping.
- **rmux:** smoke interactif Codex réel et 83 tests TUI.

La vérification finale Git propre et le SHA poussé sont volontairement effectués après la génération de ce rapport, puis rapportés séparément.

Ce qui manque encore pour 100 sur 100

Un score parfait serait mensonger sans ces preuves supplémentaires.

- **Session Vibe:** aucune session locale `Vibe`, aucun alias SSH et aucune fiche ne donne sa coordonnée. La récupération des credentials AgentiQuest `jjj` est bloquée jusqu'à réception de `host:session`. Aucun secret n'a été recherché au hasard ni affiché.
- **Providers live:** Codex est prouvé. Le smoke Claude externe a rencontré une réauthentification OAuth. Gemini n'a pas de golden path live enregistré dans cette mission.
- **Évaluations longitudinales:** les tests déterministes ferment les défauts reproduits, mais une campagne multi-run doit encore mesurer la variance réelle des agents sur plusieurs missions.
- **Migration des métadonnées de skills:** les anciens skills sans compatibilité progressive restent actifs avec warning. La prochaine étape sûre est une migration graduelle, pas une désactivation en masse.
- **Git final:** le commit et le push sont la dernière gate L0 de cette mission.

Ces limites expliquent le score 93/100. Elles sont explicites, bornées et ne diminuent pas les résultats déjà vérifiés.

Sources primaires et position finale

L'architecture suit les pratiques actuelles sans adopter le multi-agent comme dogme.

- [OpenAI, A practical guide to building AI agents](#): commencer par le pattern le plus simple qui satisfait le besoin.
- [OpenAI Agents SDK, Multi-agent orchestration](#): manager-as-tools et handoffs selon la topologie.
- [OpenAI Agents SDK, Running agents](#): tours et boucles bornés.
- [OpenAI Agents SDK, Human in the loop](#): approvals sérialisables et reprise.
- [OpenAI Agents SDK, Tracing](#): agents, tools, handoffs et guardrails comme traces.
- [Anthropic, Building effective agents](#): préférer les patterns composables et mesurables.
- [Anthropic, Demystifying evals for AI agents](#): évaluer tâche, transcript, résultat et graders.
- [MCP, Tools specification](#) et [MCP authorization](#): annotations non fiables par défaut et moindre privilège.

Position finale: les lois restent utiles. Les règles doivent être typées, scoped, provider-aware et enforceables. Le modèle récent aide à raisonner, mais le ledger, les gates, les budgets, la preuve et l'autorité restent des responsabilités du système.